

MidEng GK8.3 CI/CD Pipelines in Jenkins

Verfasser: Valdrin Aslani

Aufgabenstellung:

- Installieren von Jenkins
- Konfiguration einer einfachen Pipeline (Ausgabe "HelloWorld")
- Einbinden von einem Github/Git Repository

Aufgabe:

Installation von Jenkins:

Die Jenkins-Instanz wurde erfolgreich aufgesetzt und konfiguriert. Für dieses Projekt läuft Jenkins in einer isolierten Umgebung (Docker-Container), um eine saubere Trennung vom Host-System zu gewährleisten.

Getting Started

Getting Started

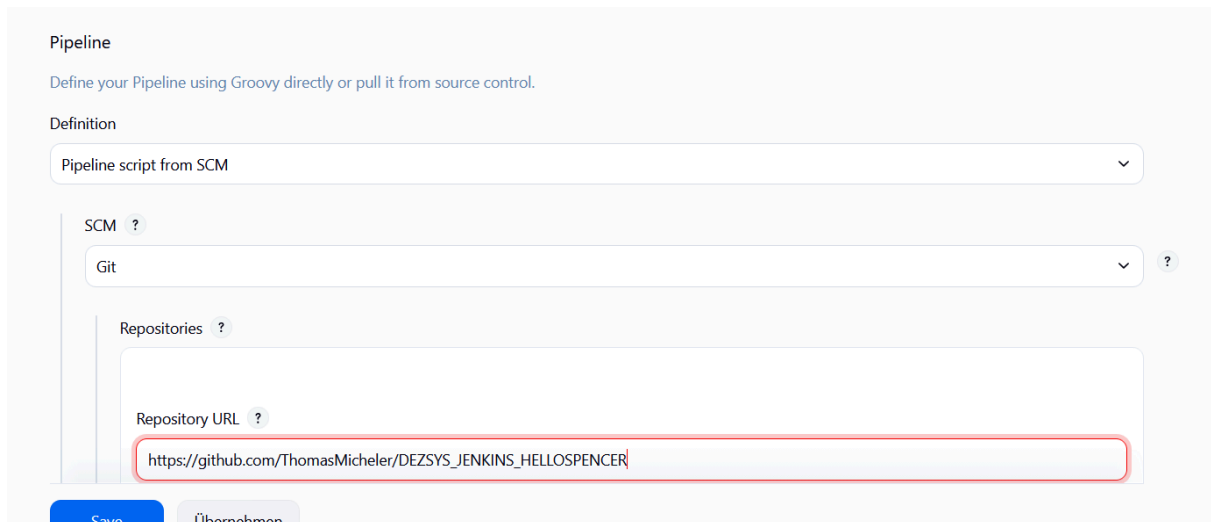
✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	✓ Credentials Binding	** Pipeline: Stage Tags Metadata
✓ Timestampers	✓ Workspace Cleanup	✓ Ant	✓ Gradle	** Pipeline: Input Step
✓ Pipeline	✓ GitHub Branch Source	✓ Pipeline: GitHub Groovy Libraries	✓ Pipeline Graph View	** Pipeline: Basic Steps
✓ Git	✓ SSH Build Agents	✓ Matrix Authorization Strategy	LDAP	** Pipeline: Declarative
⌚ Email Extension	✓ Mailer	⌚ Dark Theme		Pipeline

** Java JSON Web Token (JJWT)
** GitHub API
** Mina SSHD API :: Common
** Mina SSHD API :: Core
** Gson API
** Git client
Git
** GitHub
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
** Metrics
Pipeline Graph View
Git
** EDDSA API
** Trilead API
SSH Build Agents
Matrix Authorization Strategy
** - required dependency

Jenkins 2.555.2

Git-Einbindung:

Die Pipeline wurde so konfiguriert, dass sie nicht lokal auf dem Jenkins-Server liegt, sondern direkt versioniert aus einem Remote-Repository geladen wird (**Pipeline script from SCM**).



The screenshot shows the Jenkins Pipeline configuration interface. At the top, it says 'Pipeline' and 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, the 'Definition' dropdown is set to 'Pipeline script from SCM'. Under the 'SCM' section, 'Git' is selected. In the 'Repositories' section, the 'Repository URL' is set to 'https://github.com/ThomasMicheler/DEZSYS_JENKINS_HELLOSPENCER'. At the bottom, there are 'Save' and 'Übernehmen' buttons.

“Hello Valdrin”-Ausgabe:

Die Pipeline wurde als **Deklarative Pipeline** im `Jenkinsfile` definiert. Um Infrastruktur-Konflikte (wie fehlende lokale Docker-Daemons) zu vermeiden, nutzt die Pipeline `agent any`. Damit läuft sie stabil auf jedem verfügbaren Jenkins-Knoten.

```
pipeline {  
  agent any
```

```
  pipeline {  
    agent any  
    stages {  
      stage('Source') {  
        steps {  
          echo 'Code erfolgreich aus GitHub geladen!'  
        }  
      }  
      stage('Build') {  
        steps {  
          echo 'Build-Schritt wird ausgeführt...'  
        }  
      }  
    }  
  }  
}
```

```

    }
  }
  stage('Test') {
    steps {
      // Erfüllung des Kernkriteriums (personalisiert)
      echo 'Hello Valdrin'
    }
  }
  stage('Deployment') {
    steps {
      echo 'Deployment erfolgreich abgeschlossen!'
    }
  }
}
}

```

```

[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Hello Valdrin
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deployment)
[Pipeline] echo
Deployment erfolgreich abgeschlossen!
[Pipeline] }

```